

Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа №1
По дисциплине: «Интеллектуальный анализ данных»
Тема: «РСА»

Выполнил:
Студент 4 курса
Группы ИИ-26
Ковальчук А. И.
Проверил:
Никитин С. К.

Брест 2026

Цель работы: научиться применять метод PCA для осуществления визуализации данных.

Общее задание

1. Используя выборку по варианту, осуществить проецирование данных на плоскость первых двух и трех главных компонент (двумя способами: 1. вручную через использование `numpy.linalg.eig` для вычисления собственных значений и собственных векторов и 2. с помощью `sklearn.decomposition.PCA` для непосредственного применения метода PCA – два независимых варианта решения);
2. Выполнить визуализацию полученных главных компонент с использованием средств библиотеки `matplotlib`, обозначая экземпляры разных классов с использованием разных цветовых маркеров;
3. Используя собственные значения, рассчитанные на этапе 1, вычислить потери, связанные с преобразованием по методу PCA. Сделать выводы;
4. Оформить отчет по выполненной работе, загрузить исходный код и отчет в соответствующий репозиторий на github.

№ вариант а	Выборка	Класс
7	hcv+data.zip	Category

Код программы:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from pandas.api.types import is_numeric_dtype
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
```

```
df = pd.read_csv("../data/hcvdata.csv")
df = df.drop(columns=["Unnamed: 0"])
```

```

for col in df.columns:
    if is_numeric_dtype(df[col]):
        df[col] = df[col].fillna(df[col].median())

df["Sex"] = df["Sex"].map({"m": 1, "f": 0})

X = df.drop(columns=["Category"])
y = df["Category"]
categories = y.unique()

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

class CovPCA:
    def __init__(self, n_components=None):
        self.n_components = n_components
        self.eigenvalues = None
        self.eigenvectors = None
        self.explained_variance_ratio = None

    def fit_transform(self, X):
        if self.n_components is None:
            self.n_components = X.shape[1]

        cov_matrix = np.cov(X, rowvar=False)
        eigenvalues, eigenvectors = np.linalg.eig(cov_matrix)

        eigenvalues = np.real(eigenvalues)
        eigenvectors = np.real(eigenvectors)

        idx = eigenvalues.argsort()[::-1]
        self.eigenvalues = eigenvalues[idx]
        self.eigenvectors = eigenvectors[:, idx]

        total_variance = np.sum(self.eigenvalues)
        self.explained_variance_ratio = self.eigenvalues[:self.n_components] / total_variance

        W = self.eigenvectors[:, :self.n_components]
        X_transformed = X.dot(W)

        return X_transformed

```

```
custom_pca_2d = CovPCA(n_components=2)
custom_pca_3d = CovPCA(n_components=3)
```

```
X_custom_2d = custom_pca_2d.fit_transform(X_scaled)
X_custom_3d = custom_pca_3d.fit_transform(X_scaled)
```

```
explained_var_2d = np.sum(custom_pca_2d.explained_variance_ratio)
explained_var_3d = np.sum(custom_pca_3d.explained_variance_ratio)
```

```
loss_2d = 1 - explained_var_2d
loss_3d = 1 - explained_var_3d
```

```
print(f"Объясненная дисперсия (2D): {explained_var_2d * 100:.2f}%")
print(f"Потери при 2D-проекции: {loss_2d * 100:.2f}%\n")
print(f"Объясненная дисперсия (3D): {explained_var_3d * 100:.2f}%")
print(f"Потери при 3D-проекции: {loss_3d * 100:.2f}%")
```

```
pca_2d = PCA(n_components=2)
X_sklearn_2d = pca_2d.fit_transform(X_scaled)
```

```
pca_3d = PCA(n_components=3)
X_sklearn_3d = pca_3d.fit_transform(X_scaled)
```

```
sklearn_explained_var_2d = np.sum(pca_2d.explained_variance_ratio_)
sklearn_explained_var_3d = np.sum(pca_3d.explained_variance_ratio_)
```

```
sklearn_loss_2d = 1 - sklearn_explained_var_2d
sklearn_loss_3d = 1 - sklearn_explained_var_3d
```

```
print(f"Объясненная дисперсия (2D): {sklearn_explained_var_2d * 100:.2f}%")
print(f"Потери при 2D-проекции (sklearn): {sklearn_loss_2d * 100:.2f}%")
print(f"Объясненная дисперсия (3D): {sklearn_explained_var_3d * 100:.2f}%")
print(f"Потери при 3D-проекции (sklearn): {sklearn_loss_3d * 100:.2f}%")
```

```
fig, axes = plt.subplots(1, 2, figsize=(14, 5))
colors = ['red', 'blue', 'green', 'orange', 'purple']
```

```
for i, cat in enumerate(categories):
```

```
    idx_cat = (y == cat)
    axes[0].scatter(X_custom_2d[idx_cat, 0], X_custom_2d[idx_cat, 1],
                    c=colors[i % len(colors)], label=cat, alpha=0.7)
```

```
    axes[1].scatter(X_sklearn_2d[idx_cat, 0], X_sklearn_2d[idx_cat, 1],
                    c=colors[i % len(colors)], label=cat, alpha=0.7)
```

```

axes[0].set_title('PCA 2D (Ручное вычисление: np.linalg.eig)')
axes[1].set_title('PCA 2D (sklearn.decomposition.PCA)')
axes[0].legend()
axes[1].legend()
plt.tight_layout()
plt.show()

```

```

fig = plt.figure(figsize=(14, 6))
ax1 = fig.add_subplot(121, projection='3d')
ax2 = fig.add_subplot(122, projection='3d')

for i, cat in enumerate(categories):
    idx_cat = (y == cat)
    ax1.scatter(X_custom_3d[idx_cat, 0], X_custom_3d[idx_cat, 1], X_custom_3d[idx_cat, 2],
                c=colors[i % len(colors)], label=cat, alpha=0.7)
    ax2.scatter(X_sklearn_3d[idx_cat, 0], X_sklearn_3d[idx_cat, 1], X_sklearn_3d[idx_cat, 2],
                c=colors[i % len(colors)], label=cat, alpha=0.7)

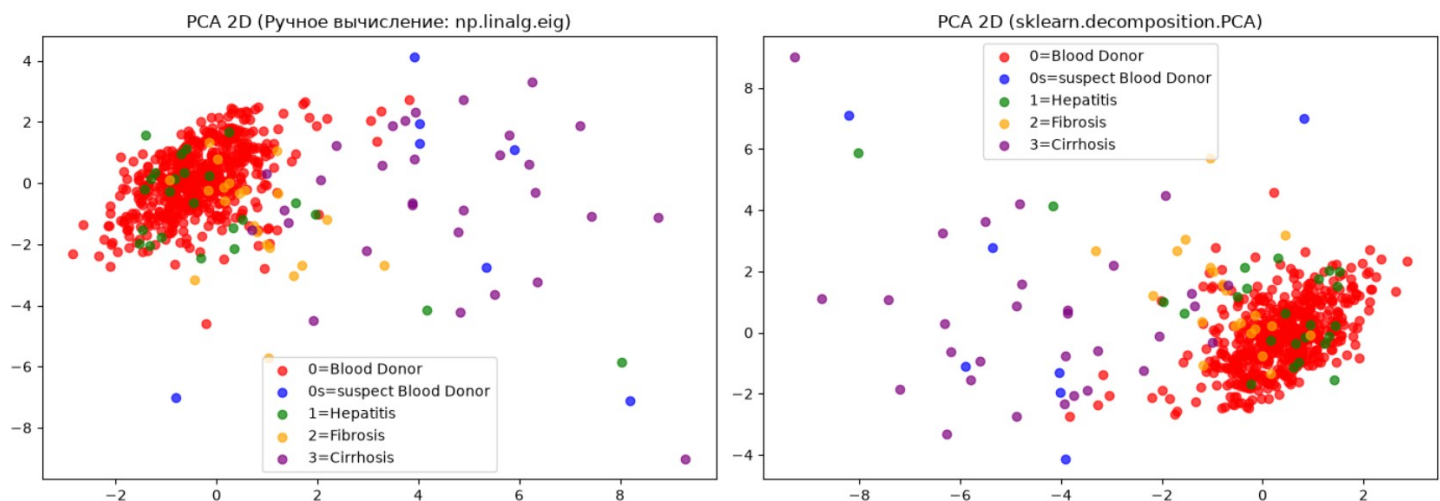
```

```

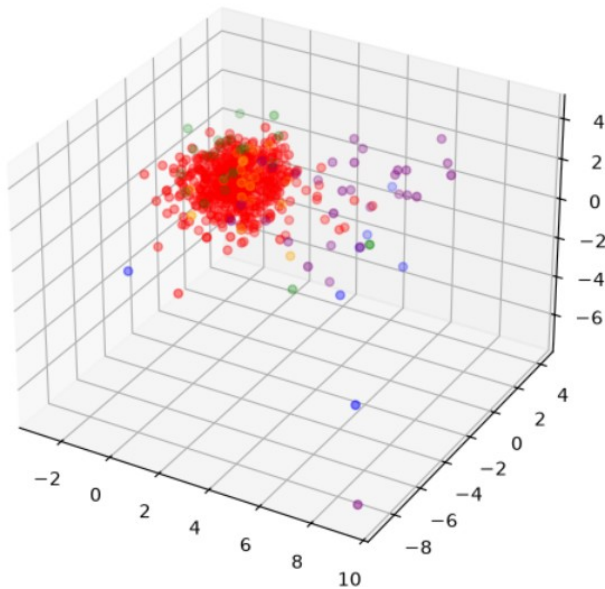
ax1.set_title('PCA 3D (Ручное вычисление)')
ax2.set_title('PCA 3D (sklearn)')
plt.show()

```

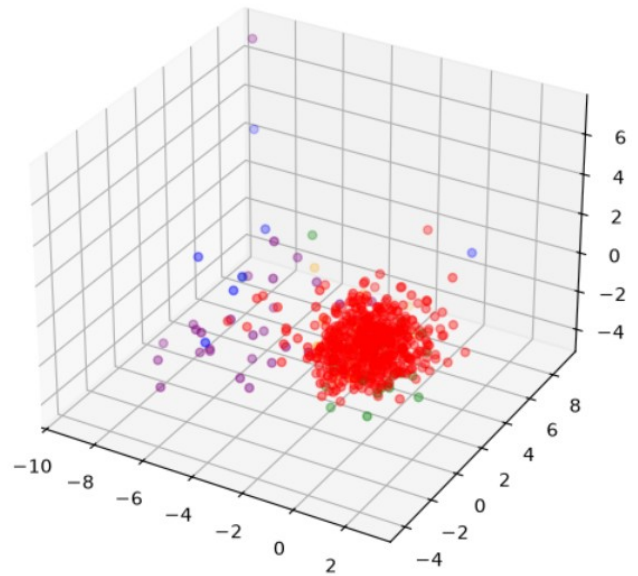
Результат программы:



PCA 3D (Ручное вычисление)



PCA 3D (sklearn)



Вывод. Метод PCA имеют следующие преимущества:

- понижение размерности с сохранением большого количества информации, что также позволяет визуализировать данные высокой размерности в двумерном или трёхмерном пространстве;
- не только позволяет значительно ускорить обучение, но и уменьшить переобучение моделей в ряде случаев;
- может использоваться для сжатия данных.

Однако не лишён и недостатков:

- неизбежная потеря части информации в данных;
- поиск только линейной зависимости в данных (в обычном PCA);
- отсутствие смыслового значения главных компонент из-за трудности их связывания с реальными признакам.